

EXERCISE 06 · UNIT 2

Parallel reduction with OpenMP

Turn four serial loops into correct, scalable parallel reductions, and learn why a shared accumulator is the classic way to get a wrong answer that looks right.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

EXERCISE

ex06-openmp-reduction

LANGUAGE

C, OpenMP

SUBMIT AS

<ROLLNO>.c

UNIT

2 — OpenMP and MPI

MARKS

10

OFFERING

Odd Semester 2026-27

What you are building

The file `starter/reduction.c` contains four loops. Each one walks an array and combines every element into a single answer: a sum, a dot product, a maximum, and a count. Combining many values into one is called a **reduction**, and it is the single most common shape in scientific computing.

All four loops currently run on one core. Your job is to make them run on all of them, and still give the right answer.

DEFINITION · REDUCTION

A computation that combines the elements of a collection into one value using an associative operator, such as `+`, `*`, `max` or `min`. Reductions parallelise well because associativity lets you combine partial results in any grouping.

The functions

You implement the four functions declared in `include/reduction.h`. Do not change that header, and do not add a `main()`: the test harness provides one.

FUNCTION	RETURNS
<code>par_sum(a, n)</code>	sum of <code>a[0..n-1]</code>
<code>par_dot(a, b, n)</code>	dot product of <code>a</code> and <code>b</code>
<code>par_max(a, n)</code>	largest element, or <code>-INFINITY</code> when <code>n == 0</code>
<code>par_count_above(a, n, t)</code>	how many elements are strictly greater than <code>t</code>

Why the obvious approach is wrong

Here is the loop most people write first.

```
double sum = 0.0;
#pragma omp parallel for /* WRONG */
for (size_t i = 0; i < n; i++) {
    sum += a[i];
}
```

`sum += a[i]` is not one instruction. It is three: read `sum`, add `a[i]`, write `sum` back. With eight threads running that sequence at once, two threads can read the same old value, each add their own element, and each write back. One update is silently lost.

PITFALL · THIS BUG DOES NOT FAIL LOUDLY

A lost update does not crash and does not print an error. It gives an answer that is slightly too small, and a different one each run. On a small array it may even be correct every time you test it, then wrong on the marked run. This is a **data race**, and it is the reason the grader runs your tests repeatedly at several thread counts rather than once.

What actually fixes it

OpenMP's `reduction` clause gives every thread a private copy of the variable, initialised to the identity of the operator (0 for `+`, 1 for `*`), and combines the copies once at the end.

```
double sum = 0.0;
#pragma omp parallel for reduction(+: sum)
for (size_t i = 0; i < n; i++) {
    sum += a[i];
}
```

The operator set includes `+`, `*`, `-`, `min`, `max`, and the bitwise and logical operators. `max` is what `par_max` wants.

COMMON MISCONCEPTION · A CRITICAL SECTION IS NOT THE FIX

Wrapping the update in `#pragma omp critical` does make the answer correct, and it also makes the loop *slower than the serial version*: every thread now queues for one lock on every single iteration, and you have paid for thread creation on top. The grader rejects a critical section inside these loops for exactly this reason. Reductions exist so you never need one here.

Floating-point addition is not associative

Adding the same numbers in a different order gives a slightly different answer, because each intermediate result is rounded. A four-thread sum groups the additions differently from a serial sum, so the last few bits will differ.

This is expected and correct. It is why the tests compare with a **relative tolerance** rather than with `==`. Do not try to force bit-exact agreement with the serial version: you would have to serialise the loop, and that defeats the exercise.

BEYOND THE SYLLABUS · WHERE THIS MATTERS IN PRACTICE

Because the grouping depends on thread count, a parallel reduction is not *reproducible* across machines by default. Codes that need bitwise reproducibility use fixed-order or compensated summation, at a cost in speed. This trade-off between speed and reproducibility comes back in Unit 4 when we look at training large models across many GPUs.

Building and testing

Run the public tests on your own machine before submitting:

```
./selfcheck.sh
```

To control the thread count:

```
OMP_NUM_THREADS=8 ./selfcheck.sh
```

If you want to compile by hand:

```
gcc -O2 -std=c11 -Wall -Wextra -Werror -fopenmp \  
-Iinclude -I../../../../_toolkit/agrade/harness \  
reduction.c tests/public.c -o t -lm  
./t
```

ON ASAICOMPUTE · RUNNING ON ASAICOMPUTE

A two-core laptop cannot show you real scaling. To see it, run on the cluster:

```
bash srun --cpus-per-task=16 --time=00:05:00 \ bash -c 'OMP_NUM_THREADS=16 ./t'
```

Try 1, 2, 4, 8, 16 threads and plot the speedup. You will see it flatten well before 16, because a reduction over a large array is limited by memory bandwidth rather than arithmetic. That flattening is the roofline model from Unit 1 showing up in your own measurement.

How this is marked

CHECK	MARKS	WHAT IT MEANS
Compiles cleanly	1	Builds with <code>-Wall -Wextra -Werror</code>
Actually uses OpenMP	1	A parallel directive and a reduction clause are present, no critical section
Public tests	1.5	The tests shipped with the exercise
Hidden tests	2.5	Same behaviour, inputs you have not seen
Deterministic across thread counts	2	Identical results over repeated runs at 1, 2 and 4 threads
Speedup against serial	2	Parallel efficiency measured against the serial reference

Speedup is graded as **efficiency**, that is speedup divided by core count, so the target adapts to whatever machine the grading runs on. Partial credit is given for partial speedup.

Submitting

Rename your finished `reduction.c` to your roll number and submit that single file:

```
AIE23001.c
```

Nothing else. No archive, no folder, no `main()`.

NOTE · BEFORE YOU SUBMIT

Run `./selfcheck.sh` at least twice with different `OMP_NUM_THREADS` values. If the answers differ between runs, you still have a race, and the grader will find it.